

Enhancing Binary Code Similarity Analysis for Software Updates: A Contextual Diffing Framework

ASIA CCS 2025

August See, Moritz Mönnich, Mathias Fischer
(Universität Hamburg, Germany)

Introduction

Function rediscovery across software updates

- Software updates
 - Function positions
 - Function layouts
 - Function structures changes
- Fixed function addresses → analysis disruption
- Locating the same **logical function** across binary versions

Previous Works

- Existing BCSA settings
 - Cross-optimization (XO) / Cross-compiler (XC) / Cross-architecture (XA) / Cross-binary (XB)
- Missing setting: **Cross-version (XV)**
- Goal: **Function rediscovery** (Rediscover the same logical function)

Problem Setting

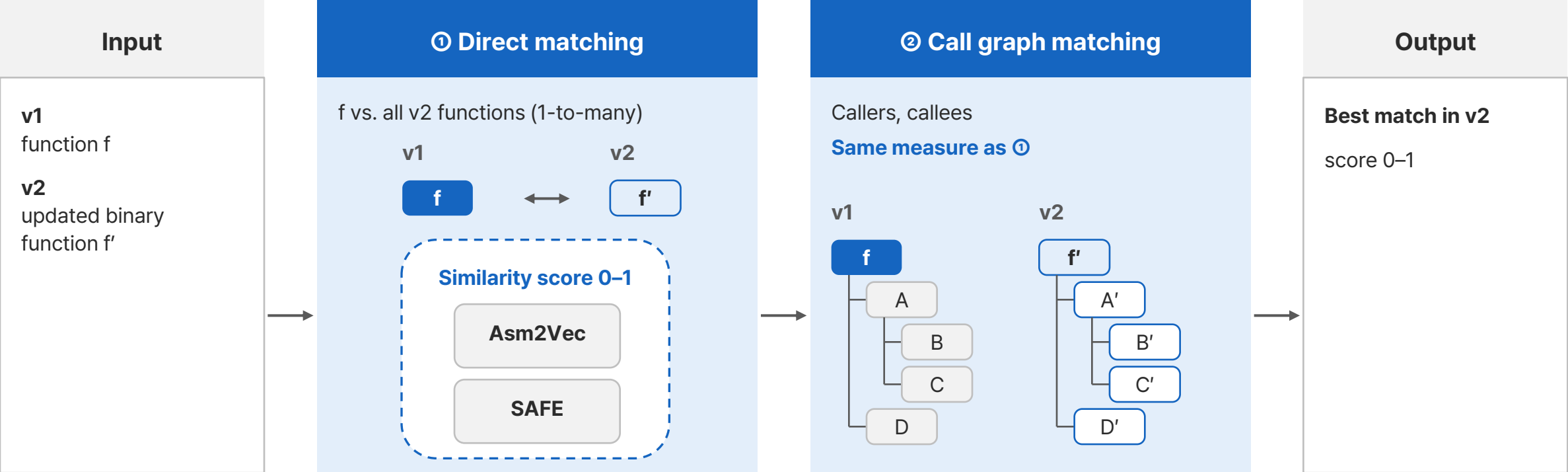
1. XV Context Stability

- Across versions:
 - function role
 - caller/callee relationships
 - mostly unchanged
- XO / XC / XB: call graph can be distorted
- XV: call graph as **contextual information**

2. XV Identical Function

- Goal: **same logical function**
- Context-based disambiguation: callers / callees

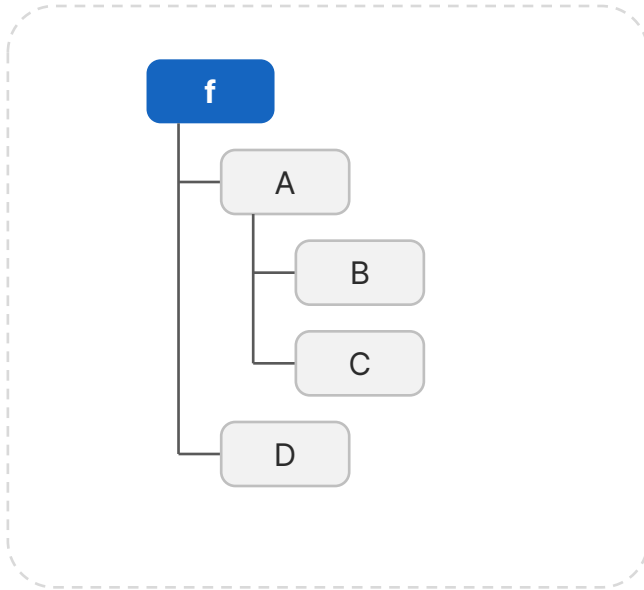
Matching Framework



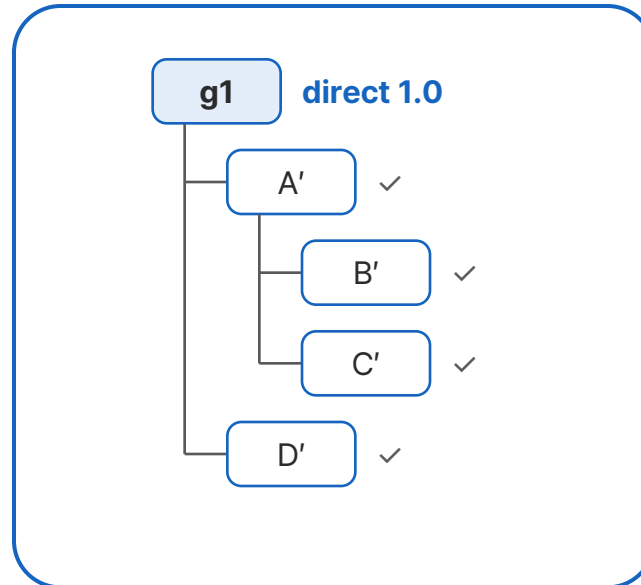
Final score =
$$\frac{\text{direct} + \text{caller} + \text{callee}}{3}$$

Call Graph Matching (CG)

Original (v1)
function to rediscover

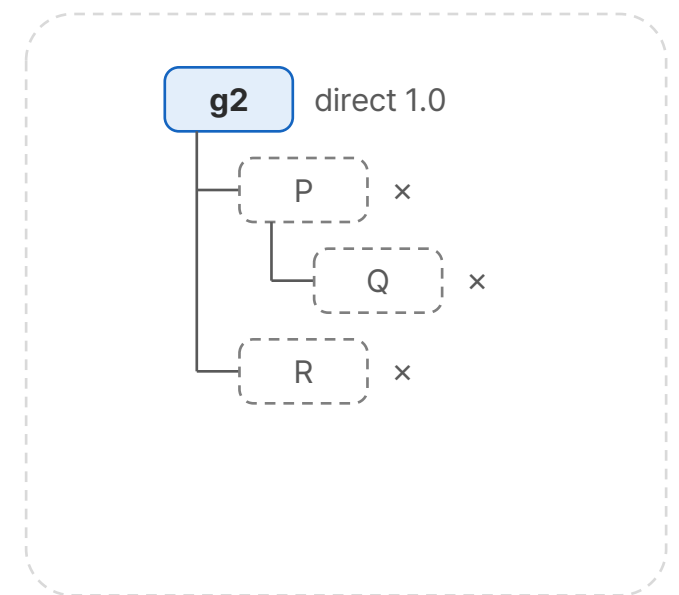


Candidate g1 (v2)



neighbors match → **high**

Candidate g2 (v2)



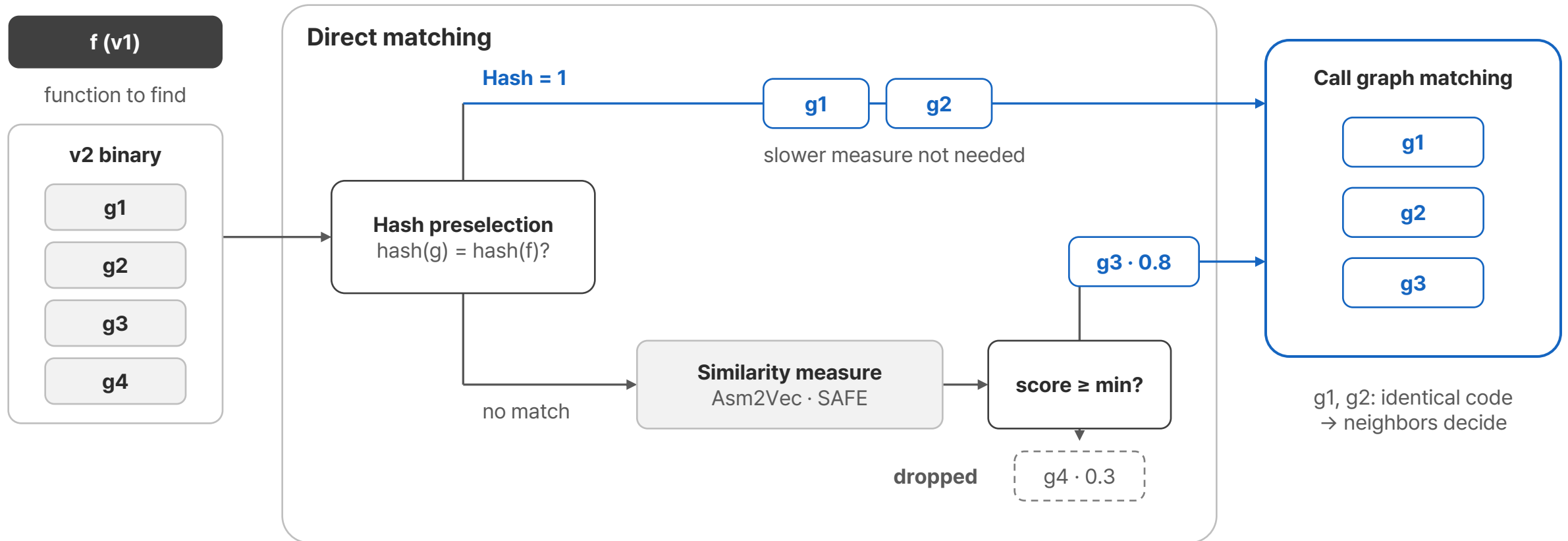
neighbors differ → **low**

CG score = direct + neighbors

Direct only: **g1 = g2 (1.0)** → with call-graph context: **g1**

CG2 (1/2): Candidate Filtering

Problem: CG compares every candidate with its callers/callees, recursively → minutes to hours per function



f = function to rediscover in v1 (original)
 g = candidate in v2 (updated)

CG2 (2/2): Call Graph Limit and Tie-Break

Left after filtering: g1, g2, g3 → each still needs a recursive neighbor comparison, and identical code ties

③ Upper bound CG

Too many callers
→ approximate instead of comparing

f (v1)



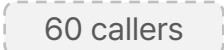
3 callers

g1 (v2)



4 ≤ 50 → compare one by one

g3 (v2)



60 > 50 → approximate

g3 caller score ≈
 $2 \cdot \min(3, 60) / (3 + 60) = 0.10$

④ Topological distance

Tie
→ compare positions in the call graph

CG score: g1 = 0.90, g2 = 0.90 → tie

f

position 12 in v1

reference

g1

position 13 in v2

$|13 - 12| = 1 \checkmark$

g2

position 40 in v2

$|40 - 12| = 28$

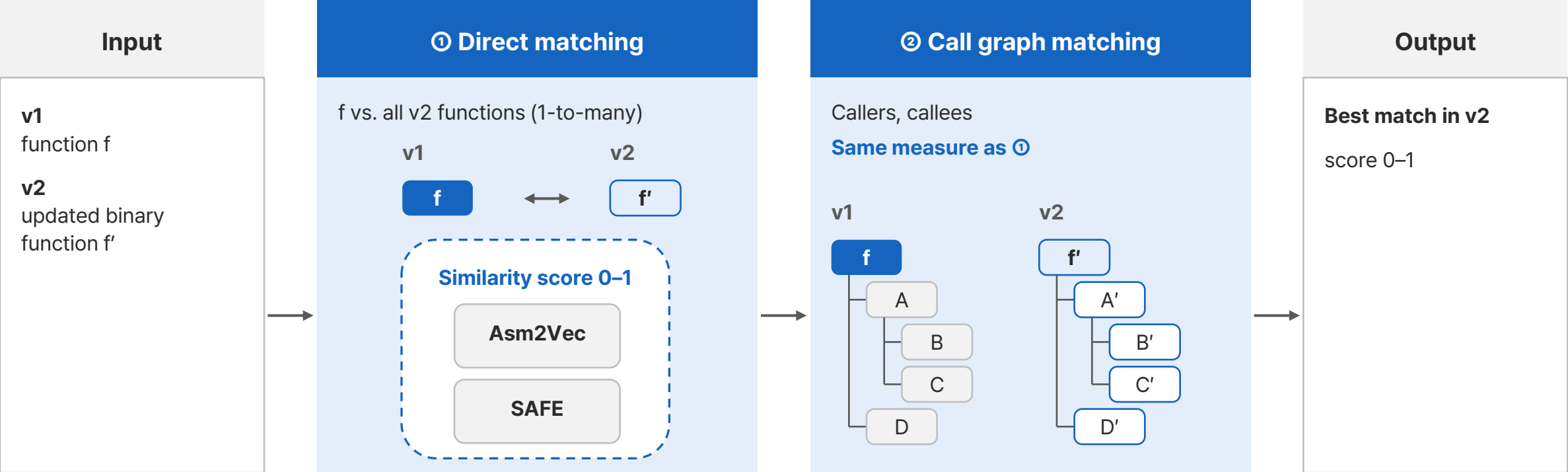
g1: distance 1
g2: distance 28

Unchanged code keeps its place
→ g1 chosen

= order from entry point (BFS)

f = function to rediscover in v1 (original)
g = candidate in v2 (updated)

Matching Framework



Final score =
$$\frac{\text{direct} + \text{caller} + \text{callee}}{3}$$

Experimental Setup

Similarity measures

- **Hash:** only detects syntactically identical functions
- **Code:** decompiled pseudocode (Ghidra), gestalt pattern matching
- **Asm2Vec, SAFE:** unofficial PyTorch implementation / pre-trained model
- **Baseline – BinDiff:** widely used reference tool that uses call graph information

Dataset

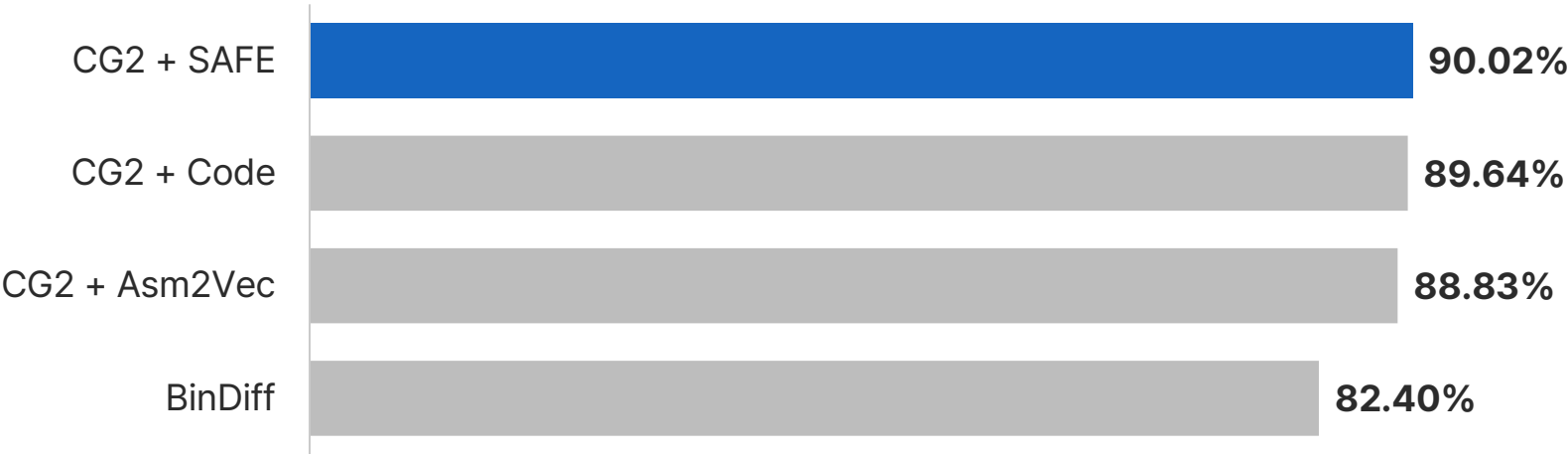
- GNU project packages, BinKit (adapted), 24,655 compiled binaries
- Latest vs. penultimate version

Ground truth

- Ground truth: same name + same source file, from debug symbols (DWARF)

RQ1: Performance Improvements

Overall function rediscovery accuracy, all configurations

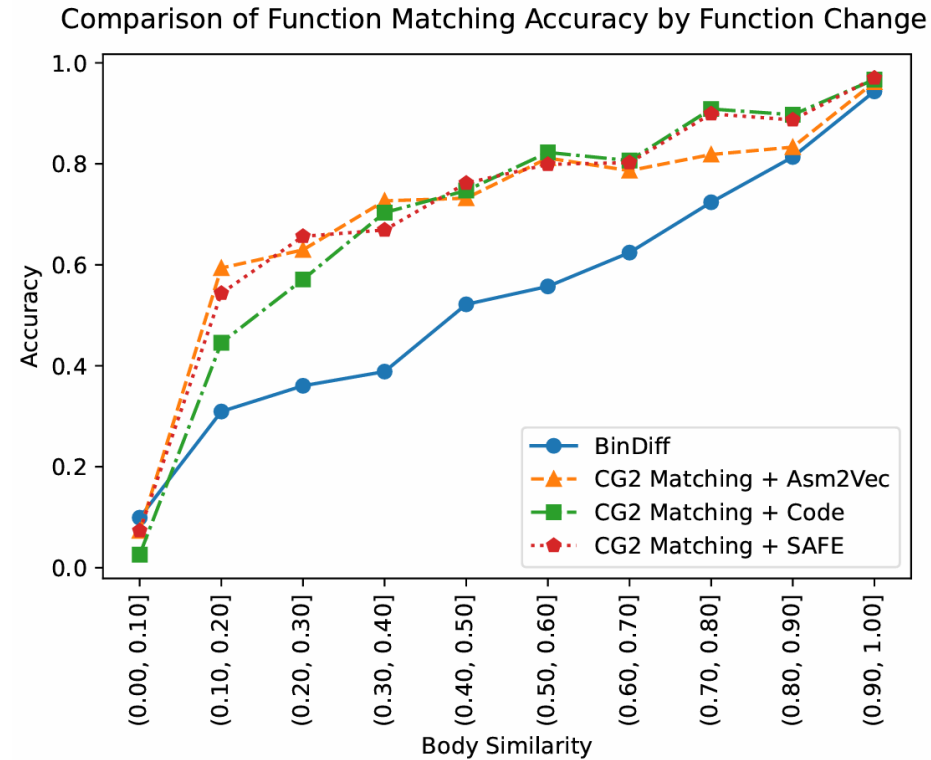


Runtime

- **CG → CG2 (SAFE):** 1584.07 ms → 330.27 ms per function (4.8× faster)

RQ2: Strengths and Weaknesses of BCSA Approaches

Body Similarity: how similar a function's instructions are between v1 and v2 (0 = completely changed, 1 = unchanged)



Limitations and Personal Thoughts

Limitations

- Assumption: generally moderate changes between two updates
- Obfuscated binaries not considered

Personal Thoughts

- Equal weights (direct + caller + callee) / 3, thresholds: 50 → no sensitivity analysis
- Tie-break assumption: topological distance assumes functions keep their position; untested under large refactoring

Thank You